

I/O Systems

Name - Saikat Sheet

Batch - 25SUB5936_Linux System Programming

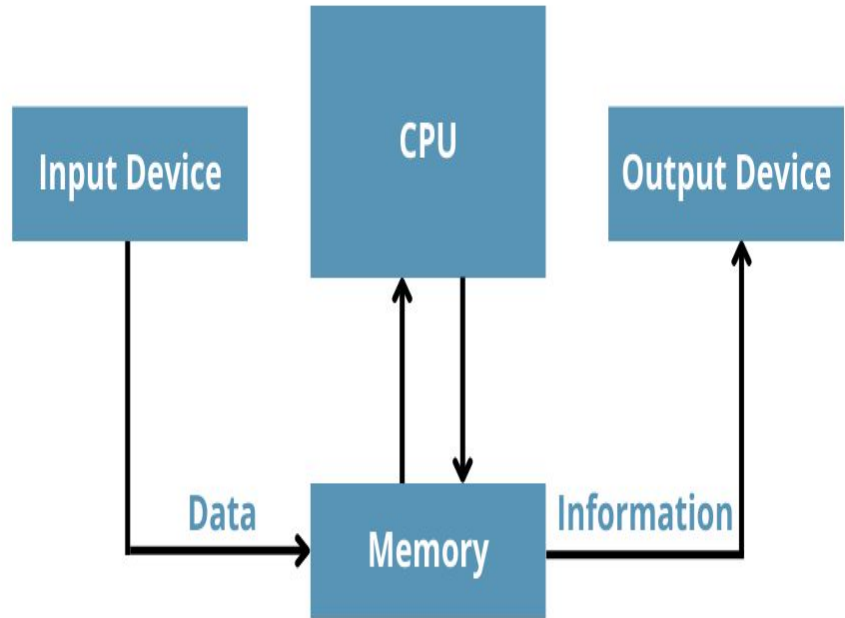
ID - 68500

Overview of I/O Systems

I/O systems manage data transfer between the computer and external devices like disks, keyboards, and networks. They utilize **Interface Circuits** and **Controllers** to bridge speed gaps. Common methods include **Programmed I/O**, **Interrupt-Driven I/O**, and **Direct Memory Access (DMA)**, which allows peripherals to bypass the CPU for faster data movement, significantly reducing processor overhead during heavy tasks

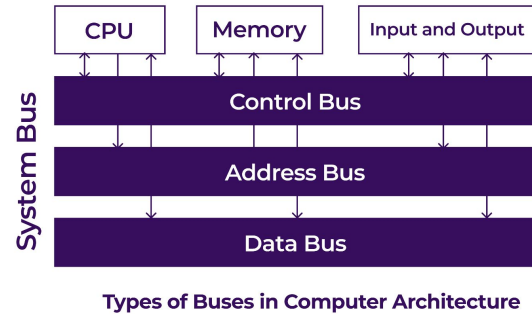
Communication Between CPU, memory, and I/O devices.

Communication occurs via the **System Bus**, consisting of data, address, and control lines. The CPU initiates transfers, while **Direct Memory Access (DMA)** allows I/O devices to move data to RAM without constant CPU supervision. **Interrupts** signal the CPU when an I/O task is complete, ensuring efficient multitasking



Introduction To Buses and Data transfer

Buses are sets of parallel wires serving as communication highways between the CPU, memory, and I/O. They consist of **Address**, **Data**, and **Control** lines. Data transfer involves the CPU placing an address on the bus, signaling a "read" or "write," and moving bits across the data lines. **Bus width** directly determines how much data transfers per cycle



Direct Memory Access (DMA) Operations

DMA allows I/O devices to transfer data directly to or from system memory, bypassing the CPU. The CPU initiates the process by sending the starting address and data count to the **DMA Controller**. Once the transfer begins, the controller manages the bus, moving data in blocks, and only interrupts the CPU once the entire task is complete.

I/O interface protocols (e.g., USB, Thunderbolt)

- **USB (Universal Serial Bus):** It is a standardized technology for connecting peripheral devices to computers, allowing for data transfer and power supply. It is known for its "plug-and-play" capability and universality, supporting devices like keyboards, phones, and hard drives.
- **Thunderbolt:** It is a high-speed hardware interface developed by Intel and Apple that combines data transfer, video signal, and power into a single cable.

Performance issues in I/O Operations and Solutions

- **Interrupts:** Instead of the CPU "asking" if data is ready (**Polling**), the device "pings" the CPU only when it's done.
- **DMA (Direct Memory Access):** A specialized controller moves data directly between I/O and RAM, letting the CPU focus on other tasks
- **Buffering:** Using a small "waiting room" (buffer) to collect data before sending it in bulk.

I/O virtualization Technologies

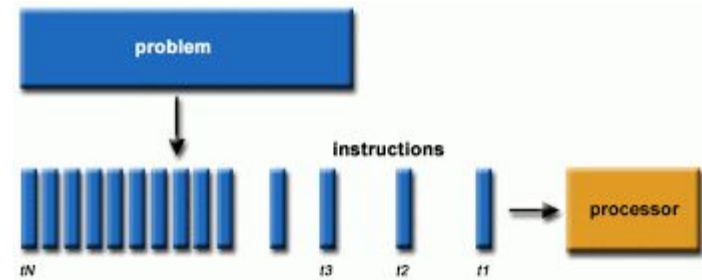
- **I/O Virtualization (IOV):** Shares physical hardware (like NICs) across multiple Virtual Machines (VMs) for better resource use
- **SR-IOV:** Splits one PCIe device into many "Virtual Functions," letting VMs talk to hardware directly to bypass software slowdowns.
- **IOMMU:** A hardware guard that maps virtual addresses to physical RAM, ensuring VMs don't crash each other.

Future Trends in I/O Devices and Systems

- **Light-Speed Data:** Instead of using slow, hot copper wires, computers will use tiny **lasers** (Optical I/O) to move data instantly using light.
- **Shared Memory:** Instead of every computer having its own "private" RAM, they will all share a giant "**community library**" of memory (CXL).

Introduction to Parallel Computing

Parallel computing is the simultaneous use of multiple processors or cores to solve a large computational problem. By breaking a task into smaller sub-tasks that run at the same time, it significantly reduces processing time. This shift from **sequential** (one-by-one) to parallel execution is essential for modern AI, weather forecasting, and high-performance gaming



Multi-Core Processors and GPUs

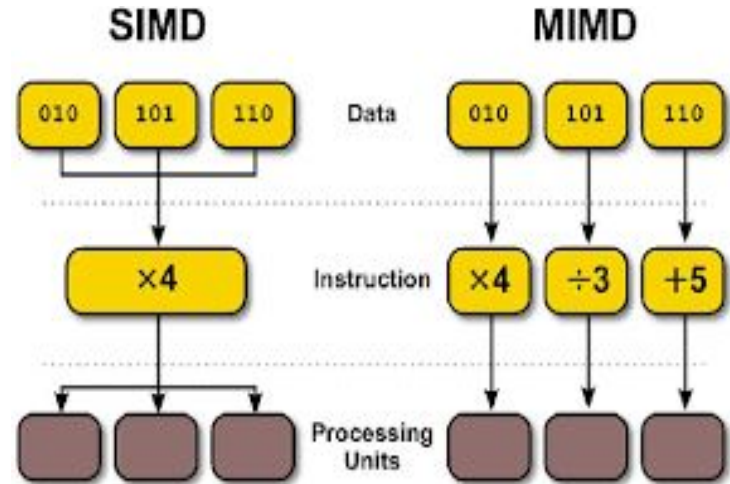
- **Multi-Core CPUs:** Feature a few powerful cores optimized for **serial tasks** and complex logic, like running an OS or single-threaded apps.
- **GPUs (Graphics Processing Units):** Contain thousands of smaller, simpler cores designed for **massive parallelism**. They excel at repetitive mathematical tasks like 3D rendering and AI model training.

Real World Applications and Examples of Multi-Core Processors and GPUs

- **AI & Deep Learning:** GPUs like the **NVIDIA** series handle massive matrix math for training LLMs, while multi-core CPUs manage data preprocessing and system logic.
- **Healthcare:** Multi-core processors run real-time robotic surgery systems, while GPUs accelerate **drug discovery** by simulating molecular interactions 30x faster than traditional methods.
- **Engineering:** Digital twins and autonomous vehicle simulations use GPUs to process high-fidelity sensor data in parallel.

Detailed Exploration of SIMD and MIMD Architectures

- **SIMD (Single Instruction, Multiple Data)** uses one controller to command many processing elements to perform the same operation on different data pieces simultaneously, like a drill sergeant leading a squad
- **MIMD (Multiple Instruction, Multiple Data)** allows each processor to act independently, executing different tasks on different data, like a team of specialized chefs.



Hardware Acceleration Through FPGAs and ASICs

- **ASICs (Application-Specific Integrated Circuits):** Permanent, custom-built chips designed for one specific task. They offer the highest speed and lowest power but cannot be changed once manufactured.
- **FPGAs (Field-Programmable Gate Arrays):** "Chameleon" chips that can be reprogrammed after BIOS updates to act like any hardware, balancing flexibility with high performance.

Challenges In Parallel Programming and Synchronization

- **Data Races:** Multiple cores accessing the same memory simultaneously, causing unpredictable errors.
- **Deadlocks:** Two or more processes waiting forever for each other to release resources.
- **Load Imbalance:** When one core does all the work while others stay idle.

Thank
you!